

Assignment 1 Report

- Kaustubh Shrivastava

Introduction

This document describes the design, development, and testing of the Array Sort Visualizer. It is a console application that sorts a user-supplied array using Bubble Sort while recording and displaying every intermediate state of the array. The program also tracks the number of inversions at each step which decrease after each iteration of the sorting process.

Design Rationale

An inversion is a pair (i, j) with $i < j$ and $a[i] > a[j]$. It measures how far an array is from ascending order. In this program, every adjacent bubble-sort swap resolves exactly one inversion, so the displayed count falls by one at each shown iteration. The total swap count therefore equals the initial inversion count for the given input.

The Top Level

- [Language/Compiler](#) - The application is written in C++ 17 and compiled with GNU g++ (GCC 13). Uses only the standard C++ library.
- [Front end](#) - A plain console based text interface without any GUI. The program reads input via cin and writes all output via cout directly to the terminal. Swap is performed one at a time and every output line displays an iteration of the sorting process.

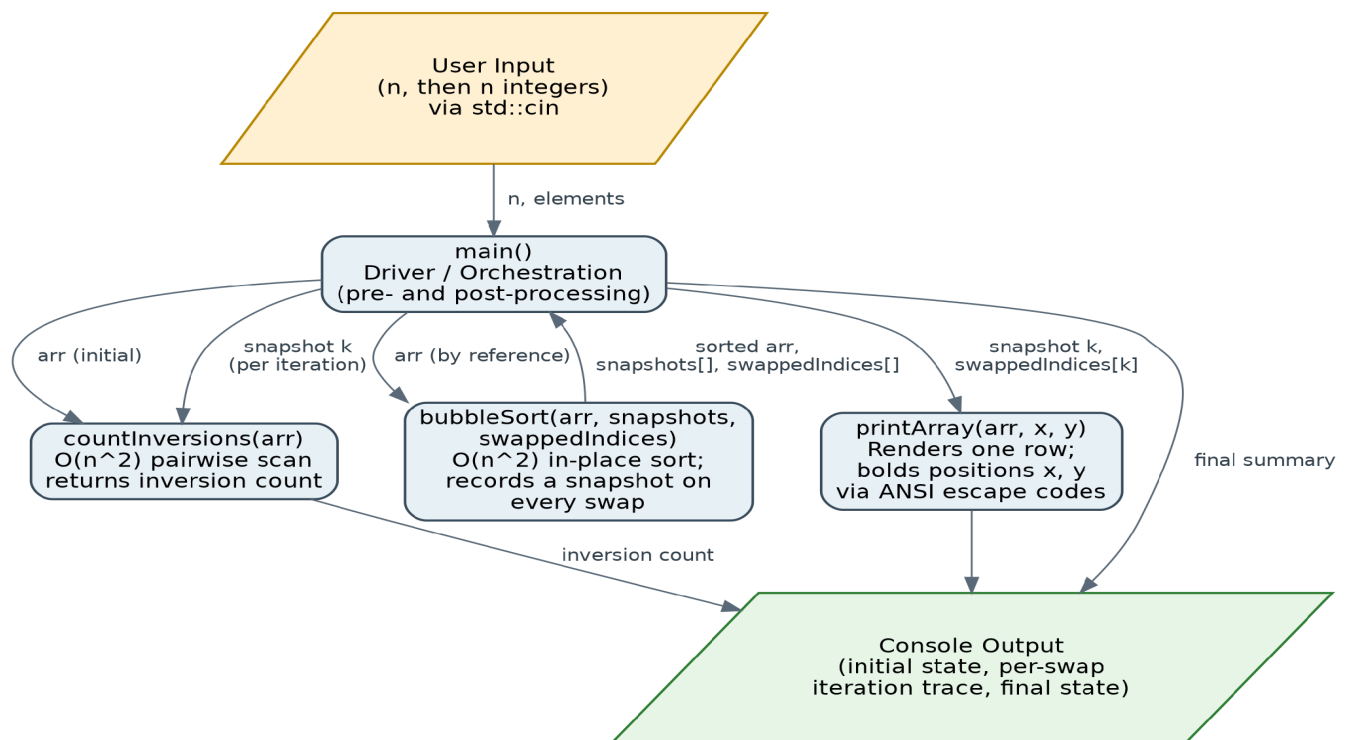
The Base

This wrapper uses a self coded and modified version of the bubble sort implementation instead of a pre-existing library routine.

- [Main Module](#) - main() function asks the user for the number of elements in the array and stores it in a variable n. Also serves as the primary module that calls other modules.
- [printArray Module](#) - printArray() function is called to output the array snapshots with recent most swapped values in bold while printing the output.

- [bubbleSort Module](#) - slightly modified implementation of the standard bubble sort algorithm. Alongside performing the typical bubble sort, this function also saves the array snapshot after a swap and records the indices of the positions that were swapped.
- [countInversions Module](#) - takes the current state of the array as an argument before every iteration and returns the inversion count. This layer does not affect the sort itself, but is used to report how the inversion count approaches 0 as the sort progresses.

Module Schema



Manufacturing

- [IDE / editor](#) - Visual Studio Code
- [Debugger](#) - GDB was used to verify the swap recording logic and the output format
- [Development Process](#) - Short and iterative. Involved coding the primary modules, compiling the file and running against a few hand-picked inputs. A simple edit-compile-run-inspect loop to suit a single-file console application of this size.
- [Testing Process](#) - Manual black-box testing after each meaningful change to the code. Involved running the compiled binary against a small fixed set of inputs (sorted,

reverse-sorted, duplicates, single element, empty, negative numbers) and visually confirm the printed inversion counts and array snapshots.

Test Cases

Example A — Reverse-sorted input (worst case, $n = 5$)

```
Enter number of elements: Enter 5 integers:  
Initial array: [ 5 4 3 2 1 ] (inversions = 10)
```

```
Iter 1: [ 4 5 3 2 1 ] inversions = 9  
Iter 2: [ 4 3 5 2 1 ] inversions = 8  
Iter 3: [ 4 3 2 5 1 ] inversions = 7  
Iter 4: [ 4 3 2 1 5 ] inversions = 6  
Iter 5: [ 3 4 2 1 5 ] inversions = 5  
Iter 6: [ 3 2 4 1 5 ] inversions = 4  
Iter 7: [ 3 2 1 4 5 ] inversions = 3  
Iter 8: [ 2 3 1 4 5 ] inversions = 2  
Iter 9: [ 2 1 3 4 5 ] inversions = 1  
Iter 10: [ 1 2 3 4 5 ] inversions = 0
```

```
Final array: [ 1 2 3 4 5 ] (inversions = 0)  
Total swap-iterations: 10
```

Example B — Random input with duplicates ($n = 8$)

```
Enter number of elements: Enter 8 integers:  
Initial array: [ 3 1 4 1 5 9 2 6 ] (inversions = 8)
```

```
Iter 1: [ 1 3 4 1 5 9 2 6 ] inversions = 7  
Iter 2: [ 1 3 1 4 5 9 2 6 ] inversions = 6  
Iter 3: [ 1 3 1 4 5 2 9 6 ] inversions = 5  
Iter 4: [ 1 3 1 4 5 2 6 9 ] inversions = 4  
Iter 5: [ 1 1 3 4 5 2 6 9 ] inversions = 3  
Iter 6: [ 1 1 3 4 2 5 6 9 ] inversions = 2  
Iter 7: [ 1 1 3 2 4 5 6 9 ] inversions = 1  
Iter 8: [ 1 1 2 3 4 5 6 9 ] inversions = 0
```

```
Final array: [ 1 1 2 3 4 5 6 9 ] (inversions = 0)  
Total swap-iterations: 8
```

Example C — Already-sorted input, best case ($n = 5$)

```
Enter number of elements: Enter 5 integers:  
Initial array: [ 1 2 3 4 5 ] (inversions = 0)
```

```
Final array: [ 1 2 3 4 5 ] (inversions = 0)  
Total swap-iterations: 0
```

Conclusion

The Array Sort Visualizer is a small, self-contained C++17 console program that layers a step-by-step visual trace and a live inversion count on top of a standard Bubble Sort implementation. Its architecture cleanly separates the sorting logic (`bubbleSort`), the analysis logic (`countInversions`), and the presentation logic (`printArray`).